

重庆山区高速公路路网安全运营综合评价系统技术文档

1. 项目概述

本项目面向重庆山区高速公路安全运营场景，将结构物状态、ETC 交通流、气象预警、交通事故、道路属性与路网拓扑等多源数据统一接入，按照“点—线—网”三级体系计算通行风险，并支持双月周期结果对比。

2. 技术栈

层次	技术与版本	用途
模型与数据处理	Python 3.12、Pandas、NumPy、OpenPyXL、python-calamine	多源数据清洗、Excel 读写和风险计算
数据库	MySQL、PyMySQL	输入数据读取、配置同步和评估结果持久化
模型服务	FastAPI 0.115+、Pydantic 2.7+、Uvicorn	参数校验、异步任务提交与状态查询
业务集成	Java 17、Spring Boot 3.4.4、RestClient、Actuator	对接 Python 模型并向业务系统暴露接口
工程与部署	Docker、Docker Compose、Maven、Git	构建、交付、服务编排和版本管理
测试	unittest、Spring Boot Test、MockRestServiceServer	Python 接口/任务测试及 Java HTTP 适配测试

Python 依赖的准确范围以 [requirements.txt](#) 为准，Java 依赖以 [java-service/pom.xml](#) 为准。

3. 项目目录

Highway_Risk_Assessment/	
├─ main.py	# Python 命令行总控入口
├─ requirements.txt	# Python 依赖
├─ Dockerfile	# Python 模型服务镜像
├─ compose.yml	# Python + Java 服务编排
├─ .env.example	# 容器环境变量模板
├─ config/	
│ └─ point_risk.ini	# 点风险参数及文件路径
│ └─ line_risk.ini	# 线风险参数及映射
│ └─ net_risk.ini	# 路网参数、拓扑和阈值
│ └─ compare.ini	# 两期对比配置
│ └─ input_db.example.ini	# 输入数据库配置模板
│ └─ output_db.example.ini	# 输出数据库配置模板
├─ data/	
│ └─ input/	# 模型输入文件
│ └─ temp/	# 中间计算文件
│ └─ output/	# 最终 Excel 成果
├─ src/	
│ └─ input/	# ETC 合并、MySQL 输入数据导出

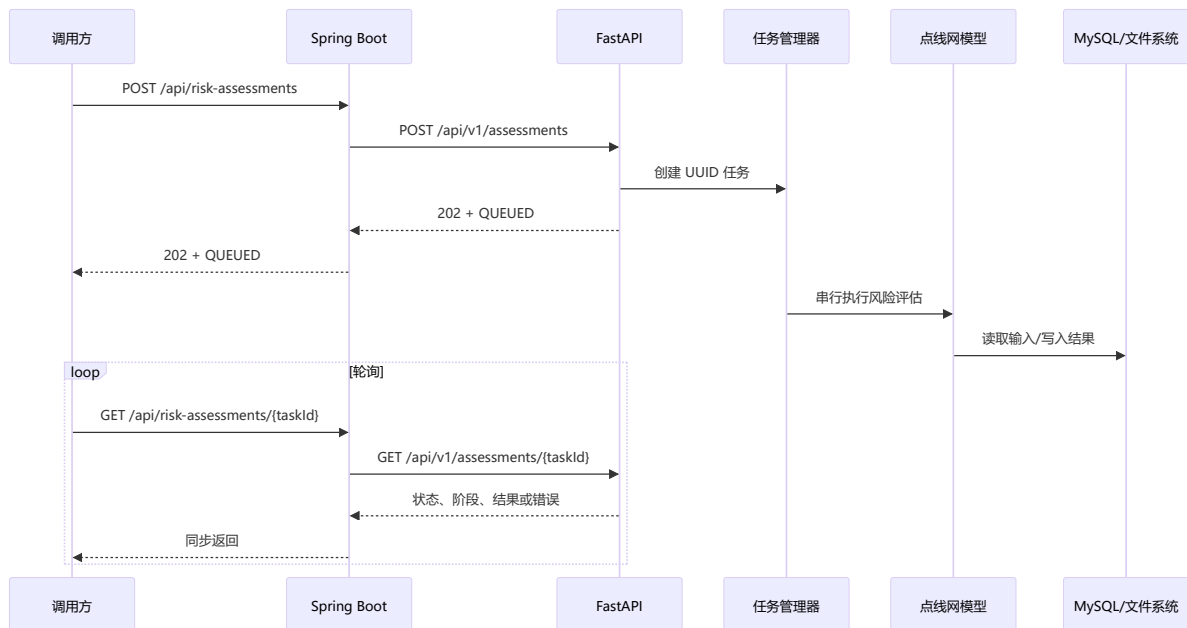
```

|   |─ config_update/          # 配置库同步
|   |─ point_risk/             # 点风险模型
|   |─ line_risk/              # 线风险模型
|   |─ net_risk/               # 网风险模型
|   |─ compare/                # 两期结果对比
|   |─ application/            # 可复用评估任务编排
|   |─ api/                    # FastAPI 与内存任务管理
|   └─ log_create/             # 日志管理
└─ java-service/               # Spring Boot 集成服务
   └─ tests/                   # Python 自动化测试

```

4. 核心业务流程

完整执行链路



应用编排器按请求参数执行以下阶段：

1. `CONFIG_UPDATE`：可选，从配置库更新 INI 文件。
2. `INPUT_PREPARATION`：可选，合并 ETC 数据并导出 MySQL 输入数据。
3. `RISK_ASSESSMENT`：可选，依次执行点、线、网评估，并在阶段间复制依赖文件。
4. `PERIOD_COMPARISON`：可选，执行两期结果对比。
5. `COLLECTING_ARTIFACTS`：收集本次任务新建或更新的成果文件。
6. `COMPLETED` 或 `FAILED`：任务结束状态。

5. FastAPI 接口说明

服务默认地址为 `http://localhost:8000`。启动后可访问 Swagger UI：

`http://localhost:8000/docs`，OpenAPI 描述：`http://localhost:8000/openapi.json`。

5.1 健康检查

```
GET /health
```

响应示例:

```
{
  "status": "UP",
  "busy": false
}
```

`busy=true` 表示至少存在一个排队或运行中的评估任务，但服务仍可接收新任务；新任务会进入串行队列。

5.2 创建评估任务

```
POST /api/v1/assessments
Content-Type: application/json
```

请求字段:

字段	类型	默认值	说明
<code>prepare_input</code>	boolean	<code>true</code>	是否执行 ETC 合并和 MySQL 输入数据导出
<code>update_config</code>	boolean	<code>false</code>	是否先从配置数据库同步 INI 配置
<code>compare</code>	boolean	<code>false</code>	是否在评估后执行两期对比
<code>recalculate</code>	boolean	<code>true</code>	是否重新执行点、线、网评估

请求示例:

```
{
  "prepare_input": true,
  "update_config": false,
  "compare": true,
  "recalculate": true
}
```

成功响应状态为 `202 Accepted` :

```
{
  "task_id": "96d57f10-763b-4d3f-9918-7e25e20d91f6",
  "status": "QUEUED",
  "phase": "QUEUED",
  "created_at": "2026-08-11T03:00:00+00:00",
  "started_at": null,
  "finished_at": null,
  "result": null,
  "error": null
}
```

5.3 查询任务状态

```
GET /api/v1/assessments/{task_id}
```

成功结果示例：

```
{
  "task_id": "96d57f10-763b-4d3f-9918-7e25e20d91f6",
  "status": "SUCCEEDED",
  "phase": "COMPLETED",
  "created_at": "2026-08-11T03:00:00+00:00",
  "started_at": "2026-08-11T03:00:01+00:00",
  "finished_at": "2026-08-11T03:20:01+00:00",
  "result": {
    "elapsed_seconds": 1200.125,
    "artifacts": [
      {
        "kind": "POINT_RISK",
        "path": "data/output/全结构点通行风险值评价表.xlsx",
        "size_bytes": 102400,
        "modified_at": "2026-08-11T03:08:00+00:00"
      }
    ]
  },
  "error": null
}
```

失败任务的 status 为 FAILED、phase 为 FAILED，错误摘要写入 error。查询不存在的任务返回 404 Not Found。

6. Spring Boot 集成说明

Java 集成服务默认监听 8080，向业务系统暴露：

方法	Java 接口	转发目标
POST	/api/risk-assessments	POST /api/v1/assessments
GET	/api/risk-assessments/{taskId}	GET /api/v1/assessments/{task_id}
GET	/actuator/health	Java 服务自身健康检查

调用路径采用 Controller → Service → RiskAssessmentClient 接口 → FastApiRiskAssessmentClient 实现。该结构隔离了业务层与具体 HTTP 客户端，便于替换实现或进行单元测试。

Java 服务配置：

环境变量	默认值	说明
SERVER_PORT	8080	Java 服务端口
RISK_API_BASE_URL	http://localhost:8000	Python 模型服务地址

环境变量	默认值	说明
<code>RISK_API_CONNECT_TIMEOUT</code>	5s	建立连接超时
<code>RISK_API_READ_TIMEOUT</code>	30s	单次 HTTP 响应读取超时

由于模型采用异步任务，提交接口只等待任务创建，不等待模型计算完成。轮询状态接口也只读取内存状态，因此 30 秒读取超时不等于模型只能运行 30 秒。

Java 的 JDK HTTP 客户端固定使用 HTTP/1.1，避免默认的明文 HTTP/2 (h2c) 升级请求与 Uvicorn 组合时出现 JSON 请求体被解析为空的问题。

Java 异常映射规则：

- 非法业务参数映射为 HTTP 400；
- Python 返回 404 时 Java 同样返回 404；
- Python 其他 HTTP 错误或服务不可达时返回 502。

7. 启动与调用

7.1 本地运行 Python 命令行

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp config/input_db.example.ini config/input_db.ini
cp config/output_db.example.ini config/output_db.ini
python main.py
```

常用命令：

```
python main.py --config-update
python main.py --compare
python main.py --compare --no-recalculate --no-input
python main.py --no-input
python main.py --simple-log
```

7.2 本地启动 FastAPI

```
uvicorn src.api.app:app --host 0.0.0.0 --port 8000 --workers 1
```

7.3 本地启动 Spring Boot

先启动 FastAPI，再执行：

```
cd java-service
mvn spring-boot:run
```

或打包后运行：

```
mvn clean package
java -jar target/risk-integration-service-1.0.0.jar
```

调用示例:

```
curl -X POST "http://localhost:8080/api/risk-assessments" \
-H "Content-Type: application/json" \
-d '{
  "prepare_input": true,
  "update_config": false,
  "compare": true,
  "recalculate": true
}'

curl "http://localhost:8080/api/risk-assessments/<task_id>"
```

7.4 Docker Compose 部署

```
cp .env.example .env
# 修改 .env 中的日期、数据库地址、账号和密码
docker compose up -d --build
docker compose ps
```

默认端口:

- Python FastAPI: 8000;
- Java Spring Boot: 8080。

Compose 将 config/、data/ 和 log/ 挂载到 Python 容器, 并将 ETC 目录只读挂载到 /ETC_Data。当 MySQL 位于宿主机时, 示例使用 host.docker.internal 访问宿主机数据库。

8. 测试与验证

8.1 Python 测试

```
python3 -m unittest discover -s tests/api_tests -p 'test_*.py'
python3 -m unittest discover -s tests/net_tests -p 'test_*.py'
```

API 测试覆盖请求默认值、非法参数、任务创建、状态查询、未知任务 404、串行执行和失败状态; 路网测试覆盖关键配置解析与校验。

8.2 Java 测试

```
cd java-service
mvn test
```

Java 测试覆盖 Service 委托行为, 以及 RestClient 的请求路径、JSON 序列化和响应反序列化。